Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Операционные среды и системное программирование»

ОТЧЕТ
к лабораторной работе №1
на тему:
«СКРИПТЫ SHELL (УГЛУБЛЕННАЯ ВЕРСИЯ)»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

(дата, подпись студента)

Проверил ассистент каф. информатики
ГРИЦЕНКО Никита Юрьевич

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Индивидуальное задание 3
2 Краткие теоритические сведения 4
3 Описание функций программы 5
4 Пример выполнения программы 7
4.1 Запуск программы и процесс выполнения 7
Вывод 10
Список использованных источников 11
Приложение А (справочное) Исходный код 12

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Написать один или несколько скриптов shell в соответствии с вариантами заданий. Можно считать, что размер окна консоли жёстко задан и не может изменяться во время выполнения (существенно для некоторых вариантов заданий). Кроме bash можно использовать другие варианты shell (zsh, csh и т.д.).

Консольная версия игры "Крестики-нолики" (для двух игроков). Дополнительно: создание скриншота – файла, содержащего текст текущего игрового поля.

2 КРАТКИЕ ТЕОРИТИЧЕСКИЕ СВЕДЕНИЯ

Скрипты shell представляют собой набор команд интерпретатора командной оболочки, записанных в текстовый файл для автоматического выполнения. Они используются для автоматизации рутинных задач, администрирования систем, обработки данных и создания простых утилит. Основные оболочки включают bash (Bourne Again SHell), sh (Bourne Shell), zsh (Z Shell), csh (C Shell). Bash является наиболее распространенным и стандартным для многих Linux-систем и macOS. Ключевые элементы скриптов включают переменные, параметры командной строки, управляющие конструкции (ветвления, циклы), арифметические вычисления, функции и вызовы внешних программ [1].

Параметры и аргументы скрипта обрабатываются через позиционные параметры ($1, $2 и т.д.) и специальные переменные. Для работы с параметрами используются команды shift для сдвига аргументов и set для установки новых значений. Также доступны параметры, такие как $* и $@, для представления всех аргументов, с различиями в обработке при использовании в кавычках [2].

Ветвления реализуются с помощью конструкций if-then-elif-else-fi и case-in-esac. Условия проверяются командами test или [ ], которые поддерживают сравнения строк, чисел, проверки файлов. Например, if [$a -eq $b]; then echo "Equal"; fi. Циклы включают for-in-do-done для итерации по списку значений, while-do-done для выполнения пока условие истинно, и until-do-done для выполнения пока условие ложно. Например, for i in 1 2 3; do echo $i; done [3].

Арифметические вычисления выполняются с помощью конструкции $((expression)) или команды expr. Например, sum=$((a + b)) или sum=$(expr $a + $b). Поддерживаются основные операции: сложение, вычитание, умножение, деление, остаток от деления.

Команды shell и вызовы внешних программ интегрируются в скрипты для расширения функциональности. Фильтры и утилиты, такие как sed (потоковый редактор для замены текста), awk (язык для обработки текста и данных), wget (загрузка файлов из интернета), grep (поиск текста), cut (вырезание частей строк), sort (сортировка), позволяют эффективно манипулировать данными. Например, sed 's/old/new/g' file.txt заменяет все вхождения "old" на "new" в файле. Скрипты объединяют эти инструменты через конвейеры (|) и перенаправления ввода-вывода (>, >>, <) для решения сложных задач.

Для игры "Крестики-нолики" потребуется использовать переменные для хранения состояния игрового поля (например, массив или строку), ветвления для проверки ходов и определения победителя, циклы для организации игрового процесса до завершения игры, а также команды вывода для отображения поля в консоли. Дополнительно, для создания скриншота игрового поля, можно перенаправить вывод в файл с помощью команды tee или простого сохранения текстового представления поля в файл.

3 ОПИСАНИЕ ФУНКЦИЙ ПРОГРАММЫ

Программа представляет собой консольную игру "Крестики-нолики" для двух игроков, реализованную в виде скрипта на языке Bash. Основная логика программы организована с помощью функций, которые отвечают за различные аспекты игрового процесса. В начале скрипта инициализируются основные переменные: BOARD – массив из 9 элементов, представляющий игровое поле, где каждый элемент соответствует клетке; PLAYER – символ текущего игрока (X или O); WINNER – переменная для хранения победителя; MOVE_COUNT – счетчик сделанных ходов; GAME_OVER – флаг завершения игры; SCREENSHOT_DIR – директория для сохранения скриншотов.

Функция clear_screen очищает экран консоли с помощью управляющих последовательностей ANSI. Это обеспечивает корректное отображение игрового интерфейса при каждом обновлении.

Функция display_board отображает текущее состояние игры. Она выводит заголовок, информацию об игроках, справочную сетку с номерами клеток от 1 до 9, текущее заполнение поля символами X и O, а также инструкции для игрока. Если игра еще не завершена, отображается приглашение для ввода хода.

Функция check_winner проверяет, достигнуто ли условие победы или ничьей. Она анализирует восемь возможных выигрышных комбинаций: три горизонтальные линии (клетки 0-2, 3-5, 6-8), три вертикальные линии (0-3-6, 1-4-7, 2-5-8) и две диагонали (0-4-8, 2-4-6). Если обнаруживается три одинаковых символа подряд, переменной WINNER присваивается символ победившего игрока. Если сделано 9 ходов и победитель не выявлен, присваивается значение DRAW (ничья). Функция возвращает 0 при обнаружении завершающего условия и 1 в противном случае.

Функция make_move обрабатывает ход игрока. Она принимает позицию от 1 до 9 в качестве аргумента, проверяет корректность ввода с помощью регулярного выражения, конвертирует в индекс массива (от 0 до 8), проверяет, что выбранная клетка свободна, затем помещает символ текущего игрока в массив BOARD. После успешного хода увеличивается счетчик MOVE_COUNT и вызывается функция check_winner для проверки условий завершения игры. Если игра завершена, устанавливается флаг GAME_OVER, в противном случае происходит смена текущего игрока.

Функция save_screenshot создает текстовый файл с текущим состоянием игрового поля. Файл сохраняется в директории screenshots с именем, содержащим временную метку. В файл записывается заголовок игры, дата и время, номер хода, текущий игрок, текущее состояние поля и результат игры (если игра завершена). После сохранения выводится сообщение с путем к файлу.

Функция display_result отображает финальное состояние игры после ее завершения. Она показывает итоговое расположение символов на поле, объявляет победителя или ничью, выводит количество сделанных ходов и

предлагает варианты дальнейших действий: начать новую игру, сохранить скриншот или выйти.

Функция new_game сбрасывает состояние игры к начальному. Она обнуляет массив BOARD, устанавливает текущего игрока в X, сбрасывает счетчик ходов и флаги завершения игры.

Функция main является основной точкой входа и управляет игровым циклом. Она непрерывно отображает игровое поле, считывает ввод пользователя и обрабатывает команды. Во время игры принимаются цифры 1-9 для хода, символ s для создания скриншота и q для выхода. После завершения игры принимаются команды n для новой игры, s для скриншота и q для выхода. Цикл продолжается до явного выхода пользователя.

4 ПРИМЕР ВЫПОЛНЕНИЯ ПРОГРАММЫ

4.1 Запуск программы и процесс выполнения

В результате выполнения программы была успешно разработана консольная версия игры "Крестики-нолики" для двух игроков, реализованная с использованием Bash-скрипта. Программа запускается в терминале командой "bash tic-tac-toe.sh" и предоставляет пошаговый интерфейс для игры двух участников, поочередно делающих ходы.

На рисунке 4.1 представлено начальное состояние игрового поля сразу после запуска программы. В этот момент поле пустое, отображается справочная сетка с номерами клеток от 1 до 9 для удобства выбора хода. В нижней части экрана находится приглашение для игрока X сделать первый ход с возможностью сохранения скриншота по команде 's'.

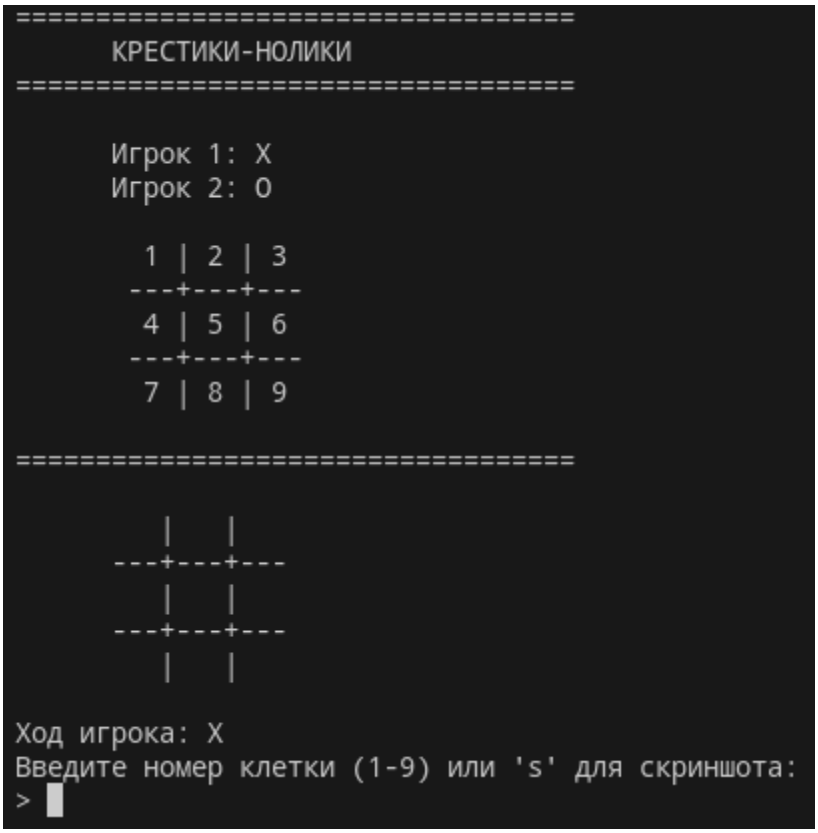


Рисунок 4.1 – Начальное состояние игрового поля

На рисунке 4.2 показан процесс игры после нескольких ходов. Видно текущее состояние поля: игрок X сделал ходы в позиции 1 и 2 (верхняя строка), игрок O сделал ходы в позиции 4 и 8. Текущим игроком является O, программа ожидает ввода номера клетки от 1 до 9. Интерфейс демонстрирует актуальное состояние доски с символами X и O, сохраняя при этом справочную нумерацию для удобства новых игроков.

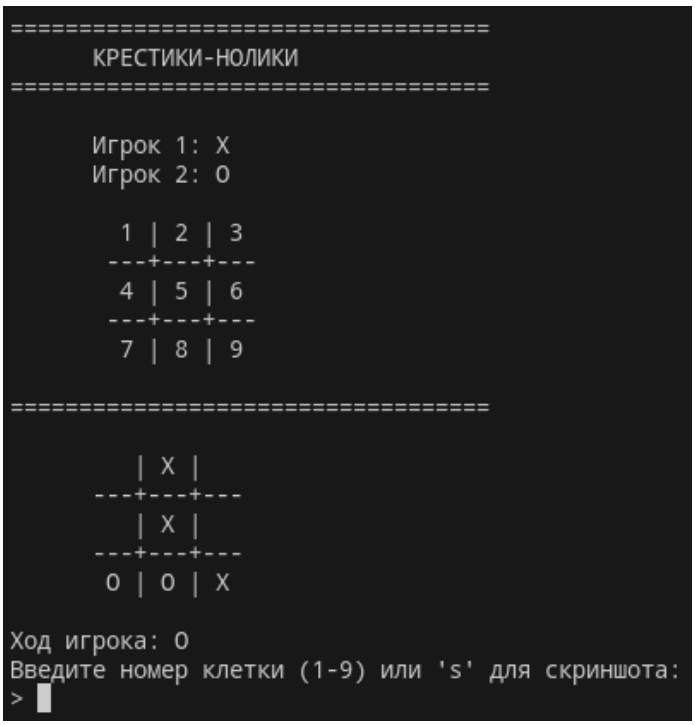


Рисунок 4.2 – Процесс игры после нескольких ходов

На рисунке 4.3 представлен пример сохраненного скриншота игры. Файл скриншота содержит полную информацию о завершенной игре: дату и время создания, номер хода, текущего игрока, состояние игрового поля и результат. В данном примере показана победная комбинация для игрока X по диагонали (позиции 1, 5, 9). Скриншоты сохраняются в текстовом формате в директории "screenshots" с уникальными именами, содержащими временную метку.

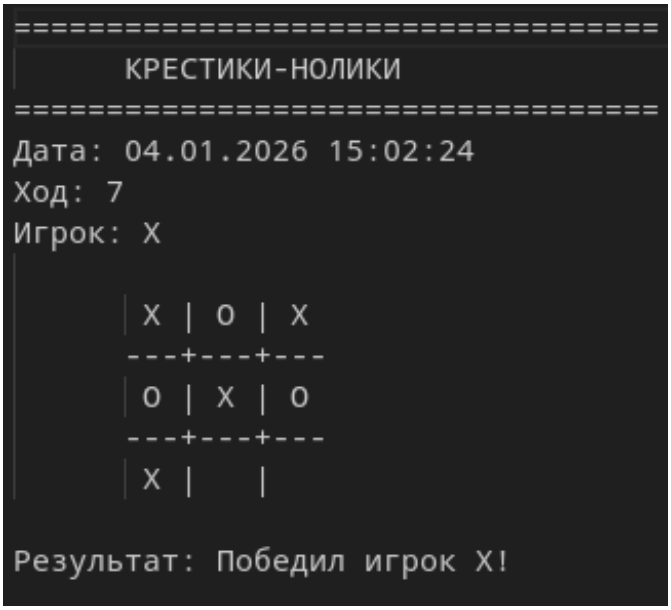


Рисунок 4.3 – Пример сохраненного скриншота игры

На рисунке 4.4 показано завершение игры с результатом "ничья". В этом случае, после заполнения всех девяти клеток, когда ни один из игроков не собрал выигрышную комбинацию, программа выводит финальное состояние поля и объявляет ничью. В этом режиме доступны три опции: начать новую игру (n), сохранить скриншот текущего результата (s) или выйти из программы (q).

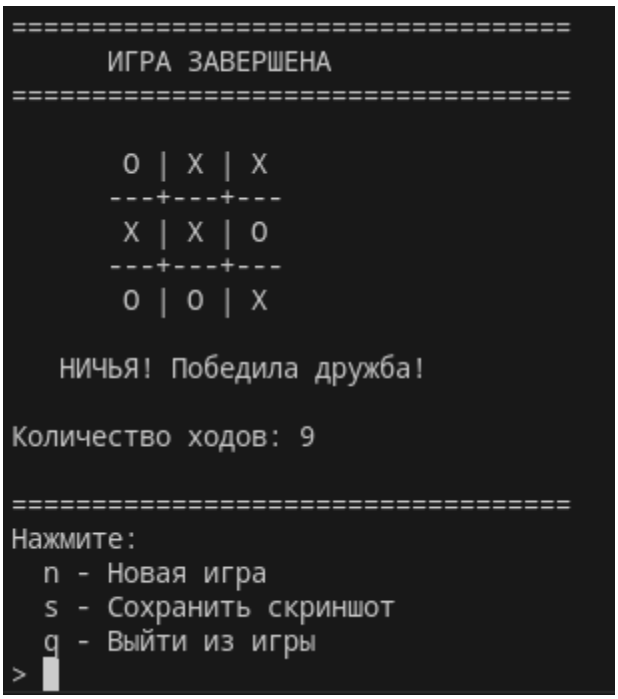


Рисунок 4.4 – Пример сохраненного скриншота игры

Программа обрабатывает различные сценарии пользовательского ввода: принимает цифры от 1 до 9 для хода, команды для создания скриншотов во время игры и после ее завершения, а также команды для управления игровым процессом. При вводе некорректных данных (например, буквы вместо цифры, занятой клетки или несуществующего номера) программа выводит соответствующее сообщение об ошибке и ожидает корректного ввода.

Игровой процесс продолжается до достижения одного из трех условий: победы одного из игроков (когда три одинаковых символа выстраиваются в линию по горизонтали, вертикали или диагонали), ничьей (когда все клетки заполнены без победной комбинации) или выхода пользователя по команде 'q'. После завершения игры предоставляется возможность сохранить итоговый скриншот или начать новую партию с очищенным полем.

ВЫВОД

В ходе выполнения лабораторной работы была разработана и протестирована консольная игра "Крестики-нолики" для двух игроков, реализованная в виде скрипта на языке Bash. Программа была создана в соответствии с требованиями задания и демонстрирует применение различных конструкций и команд shell.

Результаты тестирования показали, что разработанная программа реализует все необходимые функции классической игры "Крестики-нолики". Игра отображает игровое поле с нумерацией клеток для удобства пользователей, обеспечивает поочередный ход двух игроков (X и O), проверяет корректность вводимых данных и контролирует соблюдение правил игры. Выполнено дополнительное условие, а именно, возможность создания скриншотов текущего состояния игры в виде текстовых файлов, которые сохраняются в отдельной директории с временными метками.

Программа определяет все возможные выигрышные комбинации: три горизонтальные линии, три вертикальные линии и две диагонали. Корректно обрабатывается сценарий ничьей при полном заполнении поля без победной комбинации.

Полученные результаты демонстрируют принципы работы с основными конструкциями языка Bash: использование массивов для хранения состояния игрового поля, условные операторы для проверки победных условий, циклы для организации игрового процесса, функции для структурирования кода, работа с регулярными выражениями для валидации ввода, а также манипуляции с файлами для сохранения скриншотов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] LinuxConfig.org: Bash Scripting Tutorial: How to Write a Bash Script [Электронный ресурс]. – Режим доступа: https://linuxconfig.org/bash-scripting-tutorial. – Дата доступа: 08.02.2026.

[2] freeCodeCamp.org: Bash Scripting Tutorial – Linux Shell Script and Command Line for Beginners [Электронный ресурс]. – Режим доступа: https://www.freecodecamp.org/news/bash-scripting-tutorial-linux-shell-script-and-command-line-for-beginners/. – Дата доступа: 08.02.2026.

[3] CHARMM-GUI: Unix Tutorial [Электронный ресурс]. – Режим доступа: https://charmm-gui.org/?doc=lecture&lesson=10&module=unix. – Дата доступа: 08.02.2026.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Реализация tic-tac-toe.sh

```
#!/bin/bash

BOARD=(" " " " " " " " " " " " " " " " " ")
PLAYER="X"
WINNER=""
MOVE_COUNT=0
GAME_OVER=0
SCREENSHOT_DIR="./screenshots"

mkdir -p "$SCREENSHOT_DIR"


clear_screen() {
    printf "\033[2J\033[H"
}


display_board() {
    clear_screen
    echo "==================================="
    echo "      КРЕСТИКИ-НОЛИКИ"
    echo "==================================="
    echo ""
    echo "      Игрок 1: X"
    echo "      Игрок 2: O"
    echo ""
    echo "        1 | 2 | 3 "
    echo "       ---+---+---"
    echo "        4 | 5 | 6 "
    echo "       ---+---+---"
    echo "        7 | 8 | 9 "
    echo ""
    echo "==================================="
    echo ""
    echo "       ${BOARD[0]} | ${BOARD[1]} | ${BOARD[2]} "
    echo "      ---+---+---"
    echo "       ${BOARD[3]} | ${BOARD[4]} | ${BOARD[5]} "
    echo "      ---+---+---"
    echo "       ${BOARD[6]} | ${BOARD[7]} | ${BOARD[8]} "
    echo ""

    if [[ $GAME_OVER -eq 0 ]]; then
        echo "Ход игрока: $PLAYER"
        echo "Введите номер клетки (1-9) или 's' для скриншота:"
    fi
}


check_winner() {
    # Горизонтальные линии
    for i in 0 3 6; do
        if [[ ${BOARD[$i]} != " " && "${BOARD[$i]}" == "${BOARD[$((i+1))]}" &&
"${BOARD[$i]}" == "${BOARD[$((i+2))]}" ]]; then
            WINNER="${BOARD[$i]}"
            return 0
        fi
    done
```

```
    # Вертикальные линии
    for i in 0 1 2; do
        if [[ ${BOARD[$i]} != " " && "${BOARD[$i]}" == "${BOARD[$((i+3))]}" &&
"${BOARD[$i]}" == "${BOARD[$((i+6))]}" ]]; then
            WINNER="${BOARD[$i]}"
            return 0
        fi
    done

    # Диагонали
    if [[ ${BOARD[0]} != " " && "${BOARD[0]}" == "${BOARD[4]}" && "${BOARD[0]}"
== "${BOARD[8]}" ]]; then
        WINNER="${BOARD[0]}"
        return 0
    fi

    if [[ ${BOARD[2]} != " " && "${BOARD[2]}" == "${BOARD[4]}" && "${BOARD[2]}"
== "${BOARD[6]}" ]]; then
        WINNER="${BOARD[2]}"
        return 0
    fi

    # Проверка на ничью
    if [[ $MOVE_COUNT -eq 9 ]]; then
        WINNER="DRAW"
        return 0
    fi

    return 1
}

make_move() {
    local position=$1

    # Проверяем, что позиция в пределах 1-9
    if ! [[ $position =~ ^[1-9]$ ]]; then
        echo "Ошибка: введите число от 1 до 9"
        return 1
    fi

    # Конвертируем в индекс массива (0-8)
    local index=$((position-1))

    # Проверяем, что клетка свободна
    if [[ ${BOARD[$index]} != " " ]]; then
        echo "Клетка уже занята! Выберите другую."
        return 1
    fi

    # Делаем ход
    BOARD[index]=$PLAYER
    MOVE_COUNT=$((MOVE_COUNT + 1))

    # Проверяем на победу
    if check_winner; then
        GAME_OVER=1
        return 0
    fi

    # Меняем игрока
    if [[ $PLAYER == "X" ]]; then
        PLAYER="O"
    else
```

```
        PLAYER="X"
    fi

    return 0
}


save_screenshot() {
    local timestamp
    timestamp=$(date +"%Y%m%d_%H%M%S")
    local screenshot_file="$SCREENSHOT_DIR/game_${timestamp}.txt"

    cat > "$screenshot_file" << EOF
==================================
      КРЕСТИКИ-НОЛИКИ
==================================
Дата: $(date "+%d.%m.%Y %H:%M:%S")
Ход: $MOVE_COUNT
Игрок: $PLAYER

       ${BOARD[0]} | ${BOARD[1]} | ${BOARD[2]}
      ---+---+---
       ${BOARD[3]} | ${BOARD[4]} | ${BOARD[5]}
      ---+---+---
       ${BOARD[6]} | ${BOARD[7]} | ${BOARD[8]}

$(if [[ $GAME_OVER -eq 1 ]]; then
    if [[ $WINNER == "DRAW" ]]; then
        echo "Результат: НИЧЬЯ!"
    else
        echo "Результат: Победил игрок $WINNER!"
    fi
fi)
EOF

    echo "Скриншот сохранен: $screenshot_file"
    read -r -p "Нажмите Enter для продолжения..."
}

display_result() {
    clear_screen
    echo "==================================="
    echo "      ИГРА ЗАВЕРШЕНА"
    echo "==================================="
    echo ""
    echo "       ${BOARD[0]} | ${BOARD[1]} | ${BOARD[2]} "
    echo "      ---+---+---"
    echo "       ${BOARD[3]} | ${BOARD[4]} | ${BOARD[5]} "
    echo "      ---+---+---"
    echo "       ${BOARD[6]} | ${BOARD[7]} | ${BOARD[8]} "
    echo ""

    if [[ $WINNER == "DRAW" ]]; then
        echo "   НИЧЬЯ! Победила дружба!"
    else
        echo "   ПОБЕДИЛ ИГРОК $WINNER!"
    fi

    echo ""
    echo "Количество ходов: $MOVE_COUNT"
    echo ""
    echo "==================================="
    echo "Нажмите:"
    echo "  n - Новая игра"
```

```
    echo "  s - Сохранить скриншот"
    echo "  q - Выйти из игры"
}


new_game() {
    BOARD=(" " " " " " " " " " " " " " " " " " " " " " " " ")
    PLAYER="X"
    WINNER=""
    MOVE_COUNT=0
    GAME_OVER=0
}


main() {
    local input

    while true; do
        if [[ $GAME_OVER -eq 0 ]]; then
            display_board

            # Читаем ввод пользователя
            read -r -p "> " input

            case $input in
                [1-9])
                    if make_move "$input"; then
                        if [[ $GAME_OVER -eq 1 ]]; then
                            display_result
                        fi
                    else
                        echo "Некорректный ход!"
                        read -r -p "Нажмите Enter для продолжения..."
                    fi
                    ;;
                s|S)
                    save_screenshot
                    ;;
                q|Q)
                    echo "Выход из игры..."
                    exit 0
                    ;;
                *)
                    echo "Некорректный ввод. Используйте 1-9, s (скриншот) или
q (выход)"
                    read -r -p "Нажмите Enter для продолжения..."
                    ;;
            esac
        else
            # Игра завершена, ждем действия пользователя
            read -r -p "> " input

            case $input in
                n|N)
                    new_game
                    ;;
                s|S)
                    save_screenshot
                    ;;
                q|Q)
                    echo "Выход из игры..."
                    exit 0
                    ;;
                *)
```

```
                    echo "Некорректный ввод. Используйте n, s или q"
                    ;;
            esac
        fi
    done
}

trap 'echo -e "\n\nВыход из игры..."; exit 0' INT

main
```